

# Software Bill of Materials in Software Supply Chain Security: A Systematic Literature Review

Eric O'Donoghue\*, Yvette Hastings\*, Ernesto Ortiz\*, A. Redempta Manzi Muneza\*,

\*Montana State University, Bozeman MT, USA

**Abstract**—Software Bill of Materials (SBOMs) are increasingly regarded as essential tools for securing software supply chains (SSCs), yet their real-world use and adoption barriers remain poorly understood. This systematic literature review synthesizes evidence from 40 peer-reviewed studies to evaluate how SBOMs are currently used to bolster SSC security. We identify five primary application areas: vulnerability management, transparency, component assessment, risk assessment, and SSC integrity. Despite clear promise, adoption is hindered by significant barriers: generation tooling, data privacy, format/standardization, sharing/distribution, cost/overhead, vulnerability exploitability, maintenance, analysis tooling, false positives, hidden packages, and tampering. To structure our analysis, we map these barriers to the ISO/IEC 25019:2023 Quality-in-Use model, revealing critical deficiencies in SBOM trustworthiness, usability, and suitability for security tasks. We also highlight key gaps in the literature. These include the absence of applying machine learning techniques to assess SBOMs and limited evaluation of SBOMs and SSCs using software quality assurance techniques. Our findings provide actionable insights for researchers, tool developers, and practitioners seeking to advance SBOM-driven SSC security and lay a foundation for future work at the intersection of SSC assurance, automation, and empirical software engineering.

**Index Terms**—Software Supply Chain, Software Supply Chain Security, Software Bill of Materials, Quality-in-Use

## I. INTRODUCTION

Software systems play a critical part in modern life, supporting everything from communication to essential infrastructure. As reliance on software grows, securing these systems becomes increasingly important.

Software supply chains are integral to modern software development and encompass the components, processes, and tools involved in the development, delivery, and maintenance of software. A key element of SSCs are software dependencies [1]. In order to meet the demands of the current software development landscape, many software organizations rely on third-party dependencies. One study estimates that 85-97% of commercial software products rely on open-source dependencies [2].

Reliance on third-party dependencies introduces significant security. Effectively measuring and communicating the security of SSCs is critical for addressing these risks. In response to this need, various technologies have emerged to enhance SSC security. Among these, Software Bill of Materials (SBOM) technology has gained significant attention. SBOMs are comprehensive inventories of all software dependencies used in a specific application or system. This comprehensive inven-

tory enables software organizations to better understand and evaluate their SSCs [3].

As SSCs grow increasingly complex, SBOMs offer a promising solution for bolstering security. However, there is a paucity of studies providing comprehensive understanding of how SBOMs are applied to improve SSC security, barriers to their implementation, and possible solutions to these barriers.

As users and developers of SBOM technology, we sought to understand the state-of-the-art for SBOMs in securing SSCs. To this end, the main goal guiding our study is: **How are SBOMs, in the context of securing SSCs, being addressed in the literature?** To answer this, we explore several research questions:

- RQ1: Are SBOMs currently being used to bolster SSC security?
- RQ2: In what ways are SBOMs applied to assess and improve SSC security?
- RQ3: What barriers impact the use of SBOMs for SSC security?

To answer these questions, we conduct a systematic literature review (SLR), analyzing the current state of SBOM adoption, applications, and adoption barriers. Additionally, we take a novel approach to better understand SBOM adoption barriers. To gain insights into SBOM adoption barriers, we applied the ISO/IEC 25019:2023 Quality-in-Use (QIU) model [4] by mapping perceived barriers to its defined characteristics and sub-characteristics. This QIU framework offers a structured approach to group barriers into categories, including Beneficialness, Freedom from Risk, and Acceptability. A full breakdown of the ISO/IEC 25019:2023 is presented in [4]. Organizing barriers within this framework provides SBOM practitioners with a structured basis addressing SBOM adoption barriers.

This work offers insights into how SBOMs contribute to SSC security, identifies barriers to their use, and highlights opportunities for further research. We provide a foundation for further research and contribute to the growing effort to secure SSCs.

## II. RELATED WORK

In recent years, SBOMs have gained significant attention for addressing SSC security. However, comprehensive literature studies that analyze and consolidate the growing research on SBOMs remain scarce.

Sehgal and Ambili [5] conducted a survey focusing on the implementation of SBOMs, examining 25 papers that address SBOM generation approaches and cybersecurity automation. They provided a taxonomy of SBOM standards (SPDX, CycloneDX, SWID) and examined best practices for SBOM creation and maintenance, emphasizing automation as a key factor in effective SBOM management. Their study highlights the lack of unified standards and resource demands as significant barriers to SBOM adoption.

Jaatun et al. [6] explored the application of SBOMs specifically within critical infrastructure contexts. Their literature review, comprising 37 studies, revealed that while software supply chain attacks are increasingly common, SBOMs are seen as a promising countermeasure. The authors discuss how SBOMs can enhance risk modeling by providing visibility into component vulnerabilities, aiding in assessing potential threats to critical infrastructure. Importantly, Jaatun et al. [6] note that regulatory frameworks, such as Norwegian government directives, often “state what should be done, but not how,” underscoring the need for clearer guidance.

Zahan et al.’s work [7] complements the academic surveys above by taking a practitioner-centric view. Using a gray-literature review of 200 web artifacts, the authors distill the five most-cited benefits of SBOM adoption: dependency visibility, vulnerability triage, proactive risk management, license compliance, and competitive differentiation. Additionally they highlight the five most-cited challenges: immature tooling, lack of interoperability across SPDX/CycloneDX/SWID, unclear return-on-investment for SBOM consumers, time-consuming maintenance, and the risk of exposing proprietary information. Their work underscores a striking implementation gap: although U.S. Executive Order 14028<sup>1</sup> and subsequent OMB guidance require SBOMs for federal procurement, only 18% of the 412 organizations surveyed by the Linux Foundation report enterprise-wide SBOM practices. Zahan et al. therefore call for “interoperability, vendor cross-compatibility, and automation” as prerequisites for SBOMs at scale. By focusing on practitioner pain-points that have yet to be validated in peer-reviewed studies, their article highlights precisely the kinds of barriers that our systematic review categorizes with the ISO/IEC 25019:2023 Quality-in-Use model [4].

### III. METHODS

The methodology for this SLR adheres to the guidelines outlined by Kitchenham and Charters [8], which provide a structured approach to identifying, evaluating, and synthesizing research relevant to our research questions. Following Kitchenham and Charters [8], our research questions defined above serve as a guide for developing our search process.

#### A. Database Selection

The first step of our search process was selecting the research databases from which to retrieve relevant papers.

<sup>1</sup><https://www.nist.gov/itl/executive-order-14028-improving-nations-cybersecurity>

We chose ACM Digital Library<sup>2</sup>, Engineering Village<sup>3</sup>, and IEEE Xplore<sup>4</sup>. These databases were selected for their comprehensive collections of peer-reviewed research articles and conference proceedings, particularly in the fields of software development and security.

#### B. Search Strings

Next, we formulated search strings. In order to construct search strings we followed the PICO method (Population, Intervention, Comparison, Outcomes) [8]. The PICO method allows for a comprehensive and targeted identification of relevant literature by breaking down the key components of the study.

Using this method, we defined the following search string to ensure comprehensive coverage of the relevant literature:

**(“software supply chain” OR “software supply-chain”) AND (“software bill of materials” OR “SBOM”) AND (“security” OR “attack\*” OR “threat\*” OR “vulnerabilit\*”)**

Search string formatting was adjusted according to the requirements of each database. A detailed overview of the search strings, filters applied, and search date for each database is provided in Table I.

#### C. Selection Criteria and Paper Screening

We applied predefined selection criteria to filter papers relevant to our research questions. The full list of criteria is shown in Table II. Given that the use of SBOMs for SSC security is an emerging concept, we did not impose any publication date restrictions to capture as many relevant studies as possible.

On September 3<sup>rd</sup>, 2024, we executed the search strings across all selected databases, yielding an initial set of 97 papers. After applying the exclusion criteria, 70 papers remained for further relevance assessment.

To assess the papers, the abstract of each paper was reviewed against the inclusion criteria. If the abstract did not provide enough clarity, the full text was examined until a decision could be made. All 70 abstracts were initially reviewed by the lead author. To ensure the reliability of the inclusion decisions, two co-authors independently assessed a unique subset of the abstracts. In cases of disagreement, the paper was reviewed in full by both the lead author and co-author to reach a consensus. This screening process resulted in a final corpus of 40 papers deemed relevant to our study.

After compiling the final corpus of papers, we thoroughly reviewed each study and extracted all information related to the adoption of SBOMs for SSC security, the use of SBOM for SSC security, barriers for the use of SBOM for SSC security.

### IV. RESULTS

The increasing number of publications on SBOMs for SSC security highlights the field’s growth and potential impact (Fig.

<sup>2</sup><https://dl.acm.org/>

<sup>3</sup><https://www.engineeringvillage.com>

<sup>4</sup><https://ieeexplore.ieee.org/Xplore/home.jsp>

TABLE I: Overview of search strings and filters applied to the selected research databases.

| Database            | Search String                                                                                                                                                                                                                                                                                                                                                                            | Filters Applied                                  | Date Searched |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|---------------|
| ACM                 | [[All: “software supply chain”] OR [All: “software supply-chain”]] AND [[All: “software bill of materials”] OR [All: “sbom*”]] AND [[All: “security”] OR [All: “attack*”] OR [All: “threat*”] OR [All: “vulnerabilit*”]]                                                                                                                                                                 | Research Articles Only                           | 9/3/2024      |
| Engineering Village | (“software supply chain” OR “software supply-chain”) AND (“software bill of materials” OR “SBOM*”) AND (“security” OR “attack*” OR “threat*” OR “vulnerabilit*”)                                                                                                                                                                                                                         | Journals And Conferences Only, Remove Duplicates | 9/3/2024      |
| IEEE                | (“Full Text .AND. Metadata”:“software supply chain” OR “Full Text .AND. Metadata”:“software supply-chain”) AND (“Full Text .AND. Metadata”:“software bill of materials” OR “Full Text .AND. Metadata”:“SBOM*”) AND (“Full Text .AND. Metadata”:“security” OR “Full Text .AND. Metadata”:“attack*” OR “Full Text .AND. Metadata”:“threat*” OR “Full Text .AND. Metadata”:“vulnerabilit*”) | Journals And Conferences Only                    | 9/3/2024      |

TABLE II: Inclusion and exclusion criteria

| Exclusion                         | Inclusion                                                                                                        |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------|
| Duplicate papers                  | Published in a journal or scientific event                                                                       |
| Papers not available for download | Written in English                                                                                               |
| Extended abstract                 | Studies that explicitly or implicitly address the use of SBOM for improving SSC security                         |
|                                   | Studies that address limitations, challenges, and barriers in the implementation or use of SBOM for SSC security |

TABLE III: Overview of search results

| Database            | Initial   | After Exclusion Criteria | After Inclusion Criteria |
|---------------------|-----------|--------------------------|--------------------------|
| ACM                 | 30        | 22                       | 14                       |
| Engineering Village | 23        | 4                        | 4                        |
| IEEE                | 44        | 43                       | 22                       |
| <b>Total</b>        | <b>97</b> | <b>69</b>                | <b>40</b>                |

1). Yet, as shown in Table III, only 40 studies specifically address SBOMs for SSC security, with many offering only passing mentions. This limited focus, despite the recent U.S. executive order requiring SBOMs for government software providers, underscores the need for deeper academic investigation.

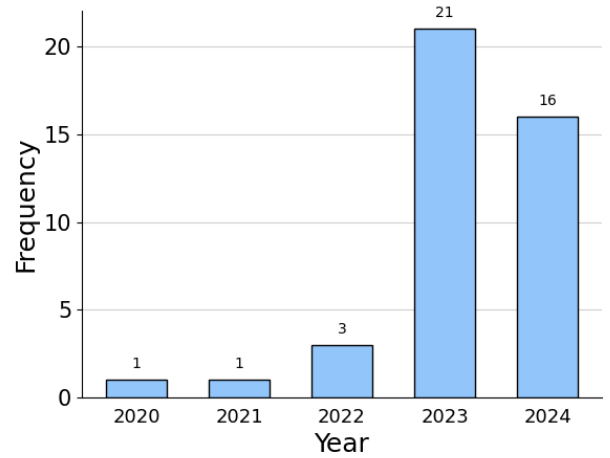


Fig. 1: Papers published per year

A. *RQ1: Are SBOMs currently being used to bolster SSC security?*

To determine whether SBOMs are being used to bolster SSC security, we analyzed publication trends, as well as literature on practitioner perspectives and SBOM adoption in open-source projects.

The rapid increase in publications that meet the inclusion criteria for this study, highlights the growing interest and adoption of SBOMs for bolstering SSC security (Fig. 1). In 2020, only a single paper addressed SBOMs in the context of SSC security, while by 2023, the number had risen sharply to 21 publications. This rise indicates both increased scholarly interest and the use of SBOM practices within the field, suggesting that SBOMs are increasingly used in the context of bolstering SSC security.

Insights from practitioners offer additional evidence of SBOMs in use for SSC security. Several studies in our review explored SBOM use through surveys and interviews with industry practitioners. For example, in a study surveying 134 developers, nearly half reported maintaining detailed SBOMs as a safeguard against SSC attacks [9]. Seventeen domain experts in the same study rated the utility of SBOMs for safeguarding against SSC attacks as moderately to highly effective, with average scores of 4.2 and median scores of 5 on a 5-point Likert scale [9]. These findings suggest that SBOMs are being employed in SSC security practices.

Other surveys provided more granular insights into SBOM usage. For instance, Stalnaker et al. found that out of 138 practitioners, 31 actively consumed SBOMs, with 20 of these using them weekly [10]. Among SBOM consumers, producers and tool makers, vulnerability tracking was a main use case (consumers: 80.7%, producers: 100%, tool makers 83.3%). These findings confirm that SBOMs are being used for security purposes, although adoption remains concentrated among a subset of informed stakeholders.

Further evidence of SBOM use is provided in an empirical study by Xia et al. [11]. The authors surveyed 65 practitioners and interviewing 17 developers. Although the study highlights significant barriers hindering widespread adoption, responses indicate that SBOMs are being incorporated into SSC security practices. Additionally, the authors cite the Linux Foundation’s SBOM and Cybersecurity Readiness report [12], which states that 90% of surveyed organizations have started or are planning their SBOM journey, with 54% already addressing SBOMs. This finding reinforces the idea that SBOMs are increasingly recognized as a component of SSC security strategies.

In another study, 16 industry practitioners were interviewed to investigate the state of SBOM adoption among stakeholders [13]. The findings suggest that SBOMs are being used, particularly by system integrators and software vendors, who see them as beneficial for enhancing security and compliance.

In addition to surveys and interviews, other work further demonstrates SBOM adoption. Nocera et al. mined GitHub and found 186 GitHub repositories that are adopting SBOMs [14]. Approximately 46% of projects analyzed had SBOMs readily accessible in their repositories or releases. While this study does not directly measure whether SBOMs are being used for security purposes, the availability of SBOMs in open-source repositories enhances transparency and enables software developers to conduct security analyses before incorporating components into their products.

Despite these encouraging signs of SBOM adoption, the majority of studies reviewed also highlighted a lack of widespread SBOM usage for SSC security, citing various barriers to adoption. These barriers not only impact the overall adoption of SBOMs but also influence how they are being used in practice. In the next subsection, we explore the specific ways SBOMs are utilized for SSC security, while a detailed discussion of barriers is presented in *subsection IV-C*.

TABLE IV: SBOM applications for assessing and improving SSC security

| Application              | Papers Mentioned                                                                                          |
|--------------------------|-----------------------------------------------------------------------------------------------------------|
| Vulnerability Management | [9], [10], [13], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26], [27], [28], [29] |
| Transparency             | [13], [26], [30], [28], [31], [32], [33]                                                                  |
| Component Assessment     | [13], [22], [24], [34], [35]                                                                              |
| Risk Assessment          | [15], [20], [21], [30], [36]                                                                              |
| SSC Integrity            | [29], [37], [38], [39]                                                                                    |
| Other                    | [13], [25], [40]                                                                                          |

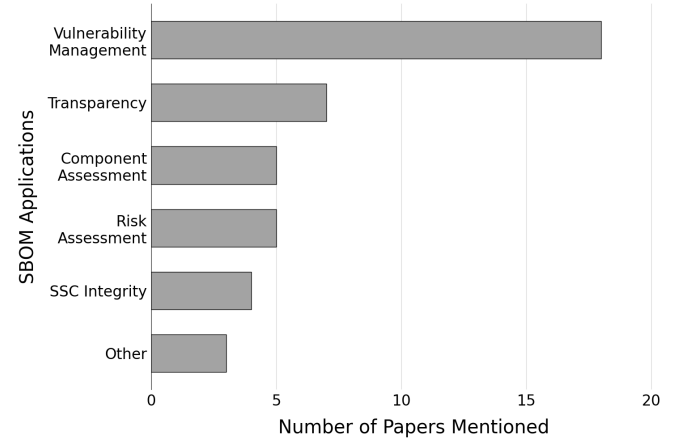


Fig. 2: Number of papers in which the use of SBOM for SSC security are mentioned.

#### B. RQ2: In what ways are SBOMs applied to assess and improve SSC security?

We identify five key applications of SBOMs for assessing and improving SSC security discussed in the literature: vulnerability management, transparency, component assessment, risk assessment, and SSC integrity (Table IV). Additionally, we grouped uses that were only mentioned once in an “Other” category. These applications demonstrate how SBOMs contribute to bolstering SSC security.

##### 1) Vulnerability Management

Vulnerability Management is the most prominent application of SBOMs for SSC security identified in the literature with 45% of the papers discussing it (Fig. 2). Vulnerability management refers to using the inventory of software components stored in an SBOM, to identify and address publicly disclosed vulnerabilities by cross-referencing component information with vulnerability databases like the National Vulnerability

Database (NVD)<sup>5</sup> [41].

Papers identified in this study vary in depth of their discussion on using SBOMs for vulnerability management. Several papers provide only a high-level acknowledgment of SBOMs role in identifying and addressing vulnerabilities, mentioning their use without offering specific implementations or detailed analyses.

For instance, Bi et al. [26] highlight that “SBOMs enhance software security by providing visibility into third-party components, enabling vulnerability management.” Similarly, Soto et al. [22] emphasize that “A comprehensive, well-maintained SBOM can help ensure software integrity by enabling organizations to identify and track potential security vulnerabilities in their software components and take appropriate action to address them.” These papers, along with, [9], [10], [13], [17], [19], [23]–[25], [28], [29], highlight the use of SBOMs for vulnerability management, even though they stop short of detailing implementations.

Other papers explore more detailed applications of SBOMs for vulnerability management, demonstrating their integration into tools, frameworks, and systems. For instance, O’Donoghue et al. [21] utilize popular SBOM analysis tools Trivy<sup>6</sup> and Grype<sup>7</sup> to identify vulnerabilities in SBOMs built from popular Docker images. SBOM analysis tools automate the process of cross-referencing component information with vulnerability databases. By analyzing SBOMs built from popular Docker images, the authors gain insights into vulnerability distributions and security risks associated with common third-party dependencies, enabling practitioners to make more informed decisions about their software components.

Extending this idea of using SBOMs as a foundation for deeper security analysis, Sun et al. [15] propose a framework for SSC security analysis that relies on SBOMs as data sources. Their approach begins with generating an SBOM and extracting its information to build a SSC knowledge graph. This knowledge graph is then enriched with known vulnerability data from public databases, enabling users to assess potential security impacts based on attributes such as CVSS scores, CWE classifications, and exploit complexities.

Taking a different technical approach, Sommer et al. [18] introduce SENTINEL, a decentralized system that combines SBOMs and blockchain technologies to automate the discovery and dissemination of security advisories. The system interfaces with vendors for identifying vulnerabilities. SENTINEL addresses two critical use cases: public vulnerability disclosure, which broadcasts security advisories to a wide audience, and private vulnerability disclosure, which ensures confidential communication of vulnerabilities using encrypted advisories and secure key distribution. This approach highlights how SBOMs can support real-time vulnerability management across diverse stakeholders.

In a more development-centric context, Mooduto et al. [27] further illustrate the practical integration of SBOMs by

integrating SBOM technology into Continuous Integration and Continuous Deployment (CICD) pipelines. The authors specifically incorporate Dependency Track<sup>8</sup>, an SBOM analysis tool, into the CICD service AWS CodePipeline<sup>9</sup>. This integration automates the identification of vulnerabilities within the SSC during the software development lifecycle. By demonstrating their method in a real-world scenario, the authors provide a practical framework for how software organizations can effectively integrate SBOM technology into their CICD pipelines to strengthen SSC security.

Furthermore, Wu et al. [20] extend the use of SBOMs to securing Internet-of-Things (IoT) devices. They emphasize that while SBOMs are static artifacts, their effective use lies in applying the information they contain to enhance SSC security. The authors propose a comprehensive scheme to assess and improve SSC security for IoT devices. Their approach begins by utilizing SBOM data to construct a component dependency map and a component querier with a Bloom filter. They then query public vulnerability databases and open-source threat intelligence platforms, similar to SBOM analysis tools, to gather relevant vulnerability information. Based on the identified vulnerabilities, they calculate risk levels and provide early warnings to users, including vulnerability information, affected components, associated security risks, and actionable recommendations.

Finally, Eggers et al. [16] explore the application of SBOMs in securing the SSCs of nuclear power plants. They propose a structured process for integrated SBOM-based vulnerability analysis, which includes continuously cross-referencing the SBOM with vulnerability databases and vendor vulnerability attestations. This approach enhances the ability of nuclear power plants to identify relevant vulnerabilities and prioritize effective risk treatment efforts. The authors provide a compelling example of SBOM utility in regards to the Log4j vulnerability, noting, “When this vulnerability was disclosed, an organization that had implemented an SBOM program with software enumerated for all installed digital devices could have readily performed a ‘where-used’ query to identify vulnerable assets, evaluate the risk, and establish risk treatments as necessary” [16].

Many of the papers discussed above demonstrate limitations and problems with the use of SBOMs for vulnerability management including false positives, lack of maturity, vulnerability dispensaries, etc. These limitations and resulting barriers to adoption are reviewed in *section IV-C*.

## 2) Transparency

Increased SSC transparency is the second most discussed application of SBOMs for SSC security. SSC transparency refers to the visibility and accessibility of all components, dependencies, processes, and relationships involved in the software development lifecycle [42], [43]. Transparency enables stakeholders to trace the origins, integrity, and security

<sup>5</sup><https://nvd.nist.gov/>

<sup>6</sup><https://www.aquasec.com/products/trivy/>

<sup>7</sup><https://github.com/anchore/grype/>

<sup>8</sup><https://dependencytrack.org/>

<sup>9</sup><https://aws.amazon.com/codepipeline/>

of software, facilitating informed decision-making, effective security management, and regulatory compliance.

SBOMs increase SSC transparency by tracking components, enabling mapping to other sources of information, and tying the component to software as it moves through the software development lifecycle [32]. By sharing SBOMs with relevant stakeholders, organizations give stakeholders a better understanding of the software’s licensing, security, and quality [13].

Several papers discuss the role of SBOMs in improving SSC transparency but provide limited practical guidance. Bi et al. [26] and Kloeg et al. [13] emphasize how SBOMs enhance visibility and foster trust among stakeholders but do not provide detailed implementation methods to achieve this. Similarly, Okafor et al. [32] highlight dependency mapping as a benefit of SBOMs but do not offer specific strategies for leveraging this approach to achieve transparency. Finally, Caven et al. [33] explain that SBOMs reduce information asymmetry to improve transparency but do not address how this can be effectively implemented in real-world scenarios.

In contrast, Xia et al. [31] present a blockchain-based solution that directly implements SSC transparency by ensuring secure and verifiable sharing of SBOMs. Their approach integrates verifiable credentials (VCs) with blockchain to enable selective disclosure, allowing stakeholders to access accurate and tamper-proof information about software components without exposing sensitive details. This method establishes trust and transparency across the SSC, enabling stakeholders to trace dependencies, verify component integrity, and ensure compliance.

### 3) Component Assessment

Component assessment involves evaluating software components for attributes such as quality, integrity, and compliance with security policies. SBOMs play a key role in enabling these assessments by providing necessary metadata.

The first theme we identify is the use of SBOM metadata to assess the quality of components. Kloeg et al. [13] briefly highlight the concept of “freshness risk” i.e. the risks posed by unmaintained components. Lee et al. [35] actualize this concept through the introduction of their tool AutoMetric. AutoMetric calculates security-related metrics for open-source components, such as inactive periods, branch protection, and the number of contributors. When used in conjunction with an SBOM, AutoMetric enables software developers to assess the quality of imported components more effectively.

Another theme is the use of SBOMs for verifying component integrity and adherence to secure build processes. Hassanshahi et al. [34] present Macaron, a framework that uses SBOMs to validate compliance with security policies like SLSA<sup>10</sup>, ensuring that components are built and maintained securely. Additionally, Soto-Valero et al. [22] briefly acknowledge the role of SBOMs in component integrity verification via checksums but do not provide implementation methods. Finally, Melo et al. [24] extend this application of SBOMs by integrating them into Zero Trust [44] architectures. Their

approach uses SBOM component data to enforce strict security controls and validate the authenticity of components within a secure framework.

### 4) Risk Assessment

Security risk assessment identifies, evaluates, and prioritizes threats to mitigate risks and enhance security [45]. SBOMs support SSC security risk assessment by building on applications such as vulnerability management and component assessment.

One approach to SSC risk assessment leverages data collected from SBOMs as input to algorithms that evaluate security risks. Wu et al. [20] propose a scheme that uses SBOM data to construct Component Dependency Trees and obtain vulnerability information as well as open source threat information. They calculate risk values based on parameters like impact range and severity, categorizing risks into four levels: low, medium, high, and super. Similarly, Sun et al. [15] develop a knowledge-driven framework that employs a SSC knowledge graph, partially built from SBOM metadata, for SSC risk assessments including component vulnerabilities and project abandonment risk.

Another approach is the use of SBOMs to analyze vulnerabilities within dependencies to inform organizations about popular dependencies that pose a significant security risk. Mounesan et al. [36] leverage SBOMs to gather dependencies of popular Docker images. Their work integrates OpenSSF Scorecard<sup>11</sup> to evaluate the security posture of these dependencies, highlighting common pitfalls and dependencies frequently associated with vulnerabilities. In a similar approach, O’Donoghue et al. [21] use SBOM analysis tools Trivy and Grype to analyze 1,151 SBOMs, identifying highly vulnerable components. These studies provide a basis for risk prioritization within the SSC.

Building on the theme of enriching SBOMs with complementary data sources for enhanced security insights, Du et al. [40] highlight how provenance reports, derived from compiler metadata, can be incorporated into SBOMs to assess risks. These reports enable end-users to identify correlations between compiler information and potential vulnerabilities, enhancing risk evaluation in software supply chains.

Finally, Gupta and Vadlamudi [30] propose a risk-based evaluation approach that integrates SBOM data with security testing results. They recommend performing Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST) on the source code of components identified in an SBOM. The resulting data, combined with SBOM metadata, is used to derive quantitative factors and calculate an overall risk score.

### 5) SSC Integrity

SSC integrity ensures that software components and their relationships within the software supply chain remain unaltered and secure [46]. SBOMs support this by providing metadata to validate the authenticity and completeness of software components.

<sup>10</sup><https://slsa.dev/>

<sup>11</sup><https://openssf.org/projects/scorecard/>

Building on this concept, researchers have explored specific ways SBOMs can help verify integrity throughout the software lifecycle. Hyeon et al. [38] propose using SBOMs with firmware/software images and manifests on a blockchain to verify file integrity and detect missing files during updates. Similarly, Ferraiuolo et al. [39] discuss using SBOMs to support reproducible builds, ensuring binaries match their source code. Martin [29] highlights a key use of SBOMs in ensuring software and SBOM information remain unaltered through cryptographic hashing and signing techniques.

Finally, Uehara et al. [37] suggest that SBOMs can detect unauthorized software infiltration by comparing component inventories, though implementation details are not provided.

While these studies present important applications for ensuring SSC integrity, they fall short of providing detailed practical implementations.

#### 6) Other Uses

Several papers discuss less common uses of SBOMs that do not fit into the primary categories identified in this study. These uses include educated supplier selection and utilizing artificial intelligence (AI).

In regard to educated supplier selection, Kloeg et al. [13] identify that SBOMs can support educated supplier selection when choosing third-party code. However, many respondents note that this approach is often impractical due to the uncertain nature of the final product at the time of software acquisition.

Ashiwal et al. [25] propose a novel use of SBOMs by leveraging AI, specifically large language models (LLMs). They use SBOMs to extract package metadata and augment it with additional information, such as vendor alert emails. This enriched data is then processed by a LLM to generate VEX data, providing detailed vulnerability information for each package in the SBOM.

#### C. RQ3: What barriers impact the effective use of SBOMs in SSC security?

Numerous barriers hinder the effective adoption and use of SBOMs for bolstering SSC security. In order to better understand these barriers, we categorize them using the ISO/IEC 25019:2023 Quality in Use (QIU) model [4]. This model provides a structured taxonomy for evaluating how software artifacts impact user experience, operational risk, and overall usefulness. Specifically, the QIU model is defined by:

- **Beneficialness:** Extent of benefit resulting from the use of a product, system or service. This characteristic includes the subcharacteristics:
  - *Usability:* Extent to which a system, product or service can be used by specified users to achieve specified goals with effectiveness, efficiency, and satisfaction in a specified context of use.
  - *Accessibility:* Extent to which products, systems, services, environments and facilities can be used by people from a population with the widest range of user needs, characteristics and capabilities to achieve identified goals in identified contexts of use.

TABLE V: SBOM adoption barriers for assessing and improving SSC security

| Barrier                      | Papers Mentioned                                                             |
|------------------------------|------------------------------------------------------------------------------|
| Generation Tooling           | [10], [11], [14], [23], [25], [26], [47], [48], [49], [50], [51], [52], [53] |
| Data Privacy                 | [10], [11], [13], [51], [23], [31]                                           |
| Format/Standardization       | [10], [11], [23], [29], [51]                                                 |
| Sharing/Distribution         | [10], [14], [23], [31], [51]                                                 |
| Cost/Overhead                | [9], [13], [23], [29]                                                        |
| Vulnerability Exploitability | [11], [24], [25]                                                             |
| Maintenance                  | [10], [29], [51]                                                             |
| Analysis Tooling             | [11], [21]                                                                   |
| False Positives              | [19], [25]                                                                   |
| Hidden Packages              | [10], [37]                                                                   |
| Tampering                    | [11], [31]                                                                   |

- *Suitability:* Extent to which behaviors or outcomes, or both, of a product meet (satisfy) specified quality requirements when used.
- **Acceptability:** Extent to which a human response is favorable when accepting or installing a product, system or service. This characteristic includes the subcharacteristics:
  - *Experience:* Extent to which users or stakeholders accumulate knowledge or skill acquired over time, especially that gained in a particular profession.
  - *Trustworthiness:* Extent to which users or stakeholders have confidence that their expectations are met in a verifiable way.
  - *Compliance:* Extent to which a user or other stakeholder has confidence that a product, system, software, or service in use meets requirements, as required by rules or law.
- **Freedom from Risk:** Extent to which a product or system mitigates the potential risk to economic status, human life, health, society, financial values, enterprise activities, or the environment. This characteristic includes the subcharacteristics:
  - *Freedom from Economic Risk:* Extent to which a product or system mitigates the potential risk to financial status, efficient operation, commercial property, reputation, or other aspects in the intended contexts of use.

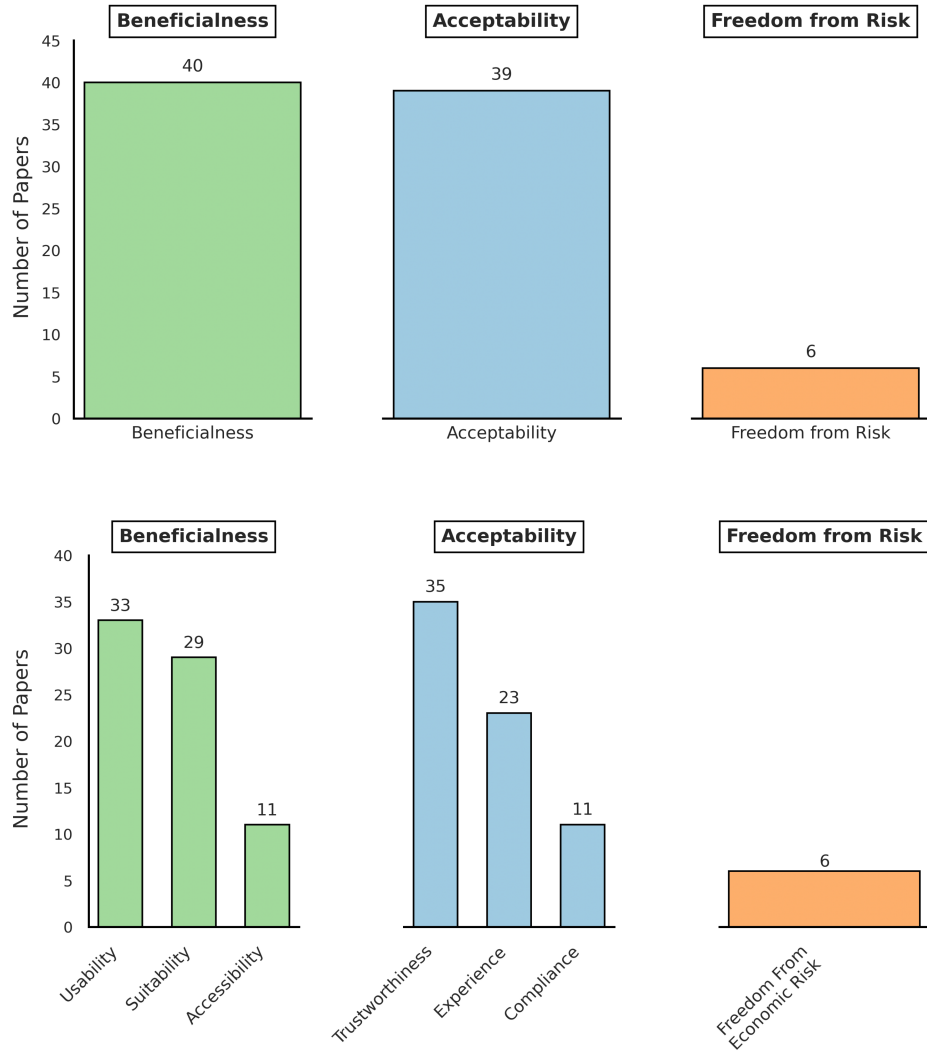


Fig. 3: Distribution of SBOM adoption barriers categorized using the ISO/IEC 25019:2023 Quality in Use (QIU) model. **Top:** number of papers aligned with each high-level QIU characteristic: Beneficialness, Acceptability, and Freedom from Risk. **Bottom:** breaks these down by subcharacteristic.

- *Freedom from Environmental and Societal Risk:* Extent to which a product or system mitigates the potential risk to the environment and society at large in the intended contexts of use.
- *Freedom from Health Risk:* Extent to which a product or system mitigates the potential risk to people's health in the intended contexts of use.
- *Freedom from Human Life Risk:* Extent to which a product or system mitigates the potential risk to people's lives in the intended contexts of use.

Organizing barriers within this framework offers a systematic lens to identify where improvements are most needed to enhance SBOM adoption and utility. In the following subsections, we discuss each identified barrier, analyzing the specific QIU subcharacteristics impacted and offering insights into their implications for SSC security. The distribution of

SBOM adoption barriers is presented in Figure 3.

#### 1) Generation Tooling

Generation tooling, which refers to the software tools used to create SBOMs, represents a significant barrier to the effective adoption and use of SBOMs in SSC security. Across the reviewed literature, there is a consistent observation that current SBOM generation tools are often immature, incomplete, and inconsistent in the quality of the artifacts they produce. These limitations impact both the **Beneficialness** and **Acceptability** dimensions of software quality-in-use.

Several papers conducted empirical evaluations of SBOM generation tooling accuracy [47]. For example, Balliu et al. [48] conducted an assessment of six SBOM generation tools targeting Java projects and observed significant discrepancies in the produced SBOMs. Tools often missed critical dependencies or produced outputs with low precision and recall. Sim-



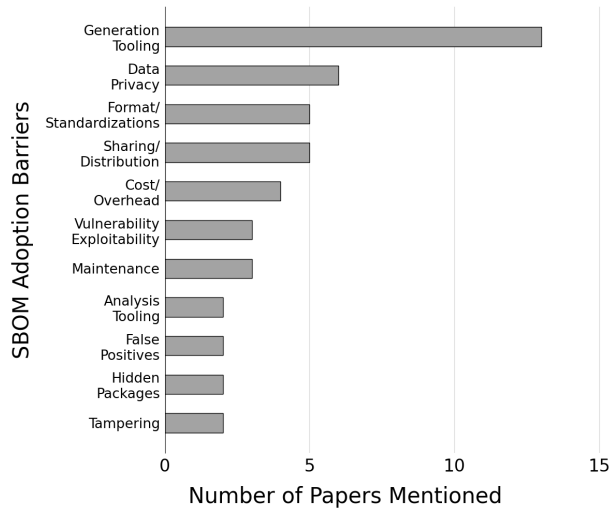


Fig. 4: Number of papers in which adoption barriers of using SBOM for SSC security are mentioned.

ilarly, Halbritter et al. [53] evaluated SBOM generation tools across web applications and system-level software, finding that no tool achieved complete and accurate SBOMs across diverse software projects. These results highlight serious challenges in achieving high *suitability* and *usability* for SBOM generation tooling.

Rabbi et al. [50] focused specifically on the npm ecosystem, comparing four SBOM generation tools. Their findings showed substantial performance variability among tools, with accuracy rates for identifying dependencies ranging widely and some tools failing to detect a significant portion of components. These inconsistencies directly impact user *trustworthiness* and further emphasize *usability* issues.

Further underscoring these concerns, Yu et al. [52] performed a differential analysis across major SBOM tools and found that metadata-based generation approaches frequently suffer from missing components and are vulnerable to parser confusion attacks, which could allow attackers to manipulate SBOM contents. Other studies reinforce these conclusions [23], [25].

Bi et al. [26] analyzed 4,786 GitHub discussions and identified widespread concerns among developers regarding incomplete and inaccurate SBOM generation. Dalia et al. [51] performed a comparative analysis of existing open-source tools and concluded that SBOM generation enabling technologies have not yet achieved full automation or maturity. In addition, Bifulco et al. [49] demonstrated that even GitHub's widely used dependency graph produces inaccurate results, further illustrating the difficulty of generating reliable supply chain metadata automatically.

Qualitative studies echo these empirical findings. Stakeholder interviews reported by Stalnaker et al. [10] indicated frustration with immature generation tooling, emphasizing that practitioners often cannot trust the outputs produced. Xia et al. [11] similarly highlighted that many SBOM tools fail

to meet minimum quality thresholds defined by initiatives such as the NTIA minimum elements list. Finally, Nocera et al. [14] observed that, despite growing awareness, SBOM tooling quality remains a limiting factor for widespread SBOM adoption among open-source projects.

Overall, the current state of SBOM generation tooling presents a major barrier to achieving the intended benefits of SBOMs for SSC security. Until tooling achieves greater *usability*, *suitability*, and *trustworthiness*, practitioners will face continued difficulties in producing accurate, reliable, and effective SBOMs.

## 2) Data Privacy

Concerns around data privacy and the inadvertent release of sensitive information through SBOMs represent a significant barrier to widespread adoption. Multiple studies have highlighted that SBOMs, by design, expose detailed component information that could potentially be weaponized by malicious actors to map attack surfaces. These concerns directly impact the QIU categories of *Beneficialness* and *Acceptability*, particularly the subcharacteristics of *trustworthiness*, *experience*, and *suitability*.

In their empirical study, Xia et al. [11] identified the dual-edged nature of transparency facilitated by SBOMs. While transparency improves supply chain visibility, it also raises practitioner fears that openly available SBOMs may become roadmaps for attackers. The study found strong practitioner support for implementing access controls and selective disclosure mechanisms, suggesting that unrestricted dissemination is generally viewed as untenable for proprietary software ecosystems.

Similarly, Stalnaker et al. [10] reported that stakeholders are concerned about SBOMs inadvertently leaking security-critical information. Several respondents in their study pointed out that publicly available SBOMs could highlight unpatched vulnerabilities or outdated components, creating significant security liabilities. The fear reflected in both of these works (Xia et al. and Stalnaker et al.) demonstrates the diminished *trustworthiness* attribute of SBOMs among stakeholders, creating hesitancy toward adoption unless secure sharing mechanisms are introduced.

Kloeg et al. [13] further reinforce these concerns from a business stakeholder perspective. Interviews revealed that organizations fear competitive disadvantages and intellectual property risks if sensitive composition information is exposed. Thus, companies are unlikely to adopt SBOMs at scale unless mechanisms for secure, controlled dissemination are established, emphasizing the need for balancing transparency with data protection requirements.

Dalia et al. [51] noted that privacy challenges stem from the lack of systematic frameworks for content tailoring and selective sharing of SBOM information. Without the ability to redact sensitive components or limit distribution to trusted parties, SBOM producers are reluctant to engage fully. They emphasize that technical solutions, such as encrypted distribution channels and signed SBOM attestations, are necessary to mitigate these risks.

The analysis by Chaora et al. [23] provides a socio-technical lens, arguing that economic incentives misalign, while consumers bear the risks of running vulnerable software, vendors bear the costs of creating and securing SBOMs. In the absence of regulatory requirements or strong market incentives, many vendors prioritize competitive secrecy over voluntary SBOM sharing, undermining broad adoption.

Finally, Xia et al. [31] proposed a blockchain-based architecture to enable secure and verifiable SBOM sharing. Their system introduces selective disclosure using verifiable credentials, allowing vendors to reveal only necessary parts of an SBOM to specific stakeholders. By offering fine-grained control over the disclosure of sensitive data, such an approach could significantly enhance *trustworthiness* and *experience* in SBOM ecosystems, addressing core barriers identified across the literature.

### 3) Format and Standardization

The lack of consistent and universally accepted SBOM formats presents a significant barrier to their effective adoption and use in SSC security. Current fragmentation among SBOM standards, divergent tooling implementations, and competing format philosophies hinder interoperability, usability, and trust in SBOMs. These issues impact the QIU categories of **Beneficialness** and **Acceptability**, specifically affecting *suitability*, *trustworthiness*, and *experience*.

Xia et al. [11] found that, despite the presence of standards like SPDX, CycloneDX, and SWID, many organizations still produce SBOMs in non-standardized, ad-hoc formats. Even when standard formats are used, there is frequent inconsistency in the inclusion of recommended minimum fields, leading to SBOMs that vary significantly in completeness and usefulness. This lack of harmonization undermines SBOM *suitability* for automated security tasks and reduces user *trustworthiness* in the generated documents.

Stalnaker et al. [10] similarly reported that the proliferation of competing SBOM standards confuses developers and stakeholders, complicating both production and consumption workflows. Their interviews revealed that differing format specifications and the absence of clear guidance on which standard to use for particular contexts (e.g., vulnerability tracking versus license management) inhibit consistent adoption. This confusion contributes to a fragmented user *experience* and weakens SBOM integration into existing development and security processes.

Chaora et al. [23] explored these socio-technical challenges through an ethnographic lens, highlighting that the absence of a single dominant format fuels industry hesitation. The uncertainty about format longevity and interoperability creates reluctance among vendors to invest heavily in SBOM production and tooling, exacerbating a cycle of limited trust and slow adoption.

Dalia et al. [51] provided a comparative analysis of current open-source SBOM tooling and standards, concluding that despite the existence of specifications, further agreement on extensions and core fields is necessary. Without these refinements, the ability of SBOMs to provide comprehensive

and actionable supply chain intelligence remains constrained, limiting their *suitability* and *trustworthiness*.

Martin et al. [29] argued that effective supply chain security depends on standardized, machine-readable, and interoperable visibility artifacts such as SBOMs. They emphasized that diverging implementations and format ambiguities must be resolved if SBOMs are to become practical foundations for trustworthy software supply chains.

Addressing format and standardization challenges is critical to realizing the full potential of SBOMs. Without clearer alignment on SBOM data models and a unified standardization effort, SBOMs will continue to suffer from low usability, inconsistent compliance with stakeholder expectations, and poor suitability for diverse SSC contexts, ultimately hindering widespread adoption.

### 4) Sharing/Distribution

The mechanisms by which SBOMs are shared and distributed constitute a major barrier to their effective adoption for SSC security. Without robust, secure, and scalable methods of distribution, SBOMs cannot achieve their goal of enhancing supply chain transparency and resilience. This barrier predominantly impacts the QIU categories of **Beneficialness** and **Acceptability**, specifically the subcharacteristics of *trustworthiness*, *suitability*, and *experience*.

Stalnaker et al. [10] found that distribution of SBOMs is currently inconsistent and underdeveloped. Their empirical study revealed that a significant number of stakeholders expect software producers to generate and distribute SBOMs alongside each release. However, there is no widely agreed-upon method or infrastructure for doing so. Participants highlighted that even when SBOMs are created, they are often not attached to release artifacts or are distributed through ad-hoc, nonstandard means. This lack of standardized distribution undermines the *trustworthiness* and *suitability* of SBOMs, making them difficult to rely upon for operational or security processes.

Chaora et al. [23] elaborated on the socio-technical barriers to SBOM sharing, emphasizing tensions between the desire for transparency and concerns over intellectual property and competitive advantage. Proprietary software vendors, in particular, were reluctant to share full SBOMs without strong assurances that sensitive information would remain protected. Moreover, differences across hosting environments (e.g., cloud services versus on-premises software) further complicate distribution, as it is not always clear when or how an SBOM should be made available. These challenges degrade the *experience* for potential SBOM consumers, who often face uncertainty about whether SBOMs will be available and trustworthy.

Dalia et al. [51] identified the lack of a dedicated, standardized, and secure platform for SBOM sharing as a core technical barrier. In practice, organizations currently rely on generic file-sharing mechanisms, which are not designed with SBOM-specific security, versioning, or privacy needs in mind. They argue that a purpose-built SBOM sharing infrastructure, incorporating encryption and secure access control, is necessary to resolve these issues. Without such infrastructure,

SBOM distribution will continue to suffer from fragmentation and usability problems, hindering its adoption.

Nocera et al. [14] indirectly corroborated these findings through their large-scale mining study of GitHub repositories. They observed that even when SBOMs are produced, only about 46% of open-source projects distribute them in any consistent or discoverable manner. This inconsistency suggests a broader systemic failure to integrate SBOM sharing into standard development and release workflows, further reducing the *usability* of SBOMs for widespread operational use.

Xia et al. [31] proposed a technical solution to some of these distribution challenges through a blockchain-based SBOM sharing architecture. By using verifiable credentials and selective disclosure mechanisms, their approach enables flexible, secure, and auditable SBOM sharing that protects sensitive information while ensuring data integrity. Their work points to future directions that could dramatically improve the *trustworthiness* of SBOM distribution, provided that such systems can achieve widespread adoption.

#### 5) Cost and Overhead

The cost and operational overhead associated with SBOM generation, maintenance, and utilization presents a key barrier to their adoption for SSC security. These burdens impact the QIU categorie **Freedom from Risk**, particularly the subcharacteristic *freedom from economic risk*.

Ladisa et al. [9] highlight that many SSC security frameworks and best practices, including SBOM initiatives, impose new process requirements on organizations without adequately reducing risk commensurately. They argue that the economic cost of implementing SBOM-related controls especially continuous monitoring and updating may not always deliver a proportional security benefit, creating a tension that organizations must manage. This tension directly impacts *economic risk mitigation*, as organizations weigh upfront and ongoing costs against uncertain downstream benefits.

Kloeg et al. [13] conducted interviews across multiple business stakeholder groups and found that perceived cost was a recurring concern. B2B customers and individual developers in particular cited concerns about who would absorb the added costs associated with SBOM generation and validation. Some B2B consumers expressed willingness to pay extra for software accompanied by SBOMs, but others feared price increases without clear value returns. On the supply side, software vendors and integrators noted that they would likely need to pass SBOM-related costs onto their customers. The introduction of explicit or hidden cost structures thus complicates SBOM adoption and may deter organizations from investing in full lifecycle SBOM management. This touches both and *freedom from economic risk* subcharacteristics.

Chaora et al. [23] broaden this concern, emphasizing that without regulatory mandates or strong industry norms, many organizations see SBOM activities as "cost centers" rather than "value centers." Their ethnographic study showed that companies, especially in non-regulated sectors, often perceive SBOM maintenance as an ongoing, indefinite overhead with little direct return, exacerbating adoption reluctance.

Martin et al. [29] argue that while visibility mechanisms like SBOMs are essential for establishing trustworthy software supply chains, the infrastructure to manage, store, distribute, and continually update SBOMs must itself be robust and cost-effective. Without streamlined, automated solutions, organizations face growing organizational burden tied to asset management, version tracking, and vulnerability monitoring all critical but resource-intensive activities.

Unless clear economic incentives or regulatory requirements are established, the cost and overhead challenges of SBOM adoption will continue to act as significant barriers, particularly impacting organizations' willingness to invest in secure SSC practices.

#### 6) Vulnerability Exploitability

One significant barrier to the effective adoption of SBOMs for SSC security is the limited ability of SBOMs to express the exploitability context of vulnerabilities. An SBOM may indicate that a vulnerable component is present, but it often lacks sufficient information to determine whether that vulnerability is actually exploitable in the QIU attributes of **Beneficialness**, particularly *suitability* and *usability*.

Xia et al. [11] observed that SBOMs currently do not systematically convey exploitability information, leaving practitioners with little guidance on whether vulnerabilities in listed components genuinely threaten their systems. Their study revealed that users often face a binary view, either a component has vulnerabilities or it does not without actionable nuance. As a result, organizations expend significant resources triaging large volumes of irrelevant vulnerability alerts, negatively affecting the perceived *usability* and practical *suitability* of SBOMs for operational security tasks.

The importance of integrating exploitability context into supply chain risk management is further emphasized by Amaral et al. [24]. They argue that SBOMs, when used alone without exploitability data, can falsely inflate the perceived risk surface, undermining Zero Trust initiatives. Their work suggests that to align SBOM usage with Zero Trust principles, it is necessary to enrich SBOM artifacts with dynamic exploitability assessments that reflect real-world use conditions.

Addressing this gap, Ashiwal et al. [25] proposed the use of Large Language Models (LLMs) to extract vulnerability information from unstructured data sources and generate Vulnerability Exploitability eXchange (VEX) documents. Their approach aims to automate the conversion of fragmented, human-readable vulnerability reports into structured, machine-readable exploitability status updates, significantly enhancing the actionable value of SBOMs. The addition of VEX data helps practitioners quickly assess whether vulnerabilities in their software supply chain components are known to affect their systems, thereby improving the *suitability* and *usability* of SBOM-based security workflows.

These studies illustrate that current SBOMs are often insufficiently informative for vulnerability prioritization without additional exploitability data. Overcoming this barrier will require broader adoption of standards like VEX, improved automation for updating exploitability status, and integration of

exploitability context into SBOM generation and consumption practices.

#### 7) Maintenance

Ongoing maintenance of SBOMs, including updating, correcting, and enriching SBOM data over time, poses a major barrier to their sustained use in SSC security. Without continuous maintenance, SBOMs quickly become outdated and lose their value as accurate representations of a software system's components and vulnerabilities. This challenge primarily impacts the QIU attributes of **Acceptability**, notably *trustworthiness* and *experience*.

Stalnaker et al. [10] noted that stakeholders perceive SBOM maintenance as a high-effort, low-reward task. Participants expressed concern that while generating an SBOM at the time of software release is already difficult, maintaining the SBOM across successive updates, patches, and dependency changes introduces significant ongoing operational overhead. When SBOMs are not kept current, users lose trust in their reliability, degrading the *trustworthiness* of the entire SBOM-based security process.

Martin et al. [29] emphasized that visibility and control over software components cannot be static. Instead, supply chain visibility must be dynamic, continuously adapting to changes in software inventory, vulnerability disclosures, and component updates. Without systematic SBOM maintenance integrated into broader asset and vulnerability management practices, organizations risk operating based on stale or incomplete supply chain information. This lack of dynamic maintenance degrades the *experience* of organizations attempting to use SBOMs for ongoing risk management.

Dalia et al. [51] reinforced the need for maintenance by pointing out that many current SBOM generation tools lack features for seamless updating or synchronization with software changes. Their analysis shows that in many cases, organizations must either regenerate full SBOMs from scratch or manually update them, both options imposing a high burden. They argue that without improvements in tooling and standards for SBOM maintenance, adoption will remain limited due to the significant *experience* penalties.

SBOMs are not a “generate once and forget” artifact. Continuous maintenance is essential for maintaining their accuracy and utility in SSC security practices. Improving automation for SBOM update workflows and embedding SBOM lifecycle management into standard development and operations pipelines will be key to overcoming this barrier.

#### 8) Analysis Tooling

Analysis tooling refers to software tools used to evaluate SBOMs for security risks, compliance gaps, or other quality attributes. Current limitations in SBOM analysis tooling pose a significant barrier to the effective use of SBOMs in SSC security, particularly impacting the QIU attributes of **Beneficialness** and **Acceptability** through reduced *trustworthiness*, *experience*, and *suitability*.

Xia et al. [11] found that SBOM analysis tooling often struggles to produce consistent and comprehensive results. Their study highlighted that different tools analyzing the same

SBOMs produced varying vulnerability findings, raising concerns about whether stakeholders can reliably use tool outputs for security decision-making. The inconsistency in analysis outcomes reduces both the *trustworthiness* of SBOM-driven risk assessments and the *suitability* of SBOMs as dependable artifacts in security workflows.

The issues with SBOM analysis tooling are starkly illustrated by O'Donoghue et al. [21], who conducted a large-scale empirical evaluation comparing two widely used SBOM static analysis tools, Trivy and Gype. Their findings revealed dramatic discrepancies: Trivy reported over seven times more vulnerabilities than Gype across 1,151 SBOMs. Trivy detected 309,022 vulnerabilities while Gype identified only 43,553. Moreover, critical vulnerabilities and overall vulnerability counts varied wildly between the tools, despite both purporting to assess the same artifacts using similar vulnerability databases. This divergence raises fundamental concerns about the *trustworthiness* of SBOM analysis tooling outputs, practitioners may either underreact (if relying on tools with poor sensitivity) or waste resources (if relying on tools prone to overreporting).

These findings also degrade the user *experience* with SBOM analysis, as practitioners cannot confidently interpret or act on results without extensive secondary validation, increasing operational burdens. Additionally, the stark inconsistency between tools undermines the perceived *suitability* of SBOMs for fulfilling their security enhancement goals, since SBOMs cannot realize their value if post-generation analysis is unreliable.

Until SBOM analysis tooling achieves greater consistency, accuracy, and transparency, the full potential of SBOMs in SSC security will remain unrealized. Building confidence in analysis outputs must be a priority for the SBOM ecosystem to mature.

#### 9) False Positives

The prevalence of false positives during vulnerability analysis of SBOMs represents a notable barrier to their effective use in SSC security. High rates of false positives degrade the QIU attributes of **Beneficialness** and **Acceptability**, particularly impacting *trustworthiness*, and *usability*.

Bottner et al. [19] evaluated two open-source Software Composition Analysis (SCA) tools and found that both suffered from significant false positive rates. Their findings emphasize that analysis tools often incorrectly report vulnerabilities that are not actually exploitable in the deployed product. This undermines user trust in SBOM-based risk assessments and imposes additional burdens on security teams to manually triage irrelevant findings. Ashiwal et al. [25] further highlighted that false positives are exacerbated by incomplete or delayed vulnerability information in structured databases like the NVD.

#### 10) Hidden Packages

Hidden or undisclosed software packages present a serious barrier to the effective use of SBOMs for SSC security. When SBOMs fail to capture all direct and transitive dependencies, they undermine the QIU attributes of **Beneficialness** and

**Acceptability**, particularly *suitability*, *usability*, and *trustworthiness*.

Stalnaker et al. [10] highlighted that incomplete SBOMs often miss manually installed, dynamically loaded, or deeply nested dependencies. Stakeholders reported frustration when critical components were omitted, preventing accurate vulnerability assessments. This incomplete visibility damages trust in SBOMs and limits their practical usability. Bottner et al. [37] further emphasized the challenge of hidden packages through their evaluation of SCA tools. Their study showed that even sophisticated tools frequently miss transitive dependencies, especially when metadata is missing or build configurations are complex. Such omissions cause significant gaps in SBOM coverage, limiting their *suitability* for securing modern software systems.

#### 11) Tampering

The risk of tampering with SBOMs represents a fundamental barrier to their secure use in SSC security. If SBOMs are modified after generation, such as by removing components to hide vulnerabilities, their trustworthiness and utility are compromised. This barrier primarily impacts the QIU attribute of **Freedom from Risk**, particularly *economic risk mitigation*.

Xia et al. [11] emphasized that without proper integrity mechanisms, SBOMs could be altered to misrepresent software composition, exposing consumers to unseen vulnerabilities. Their study stressed the need for digitally signing SBOMs to ensure authenticity and to detect tampering, but observed that current practice often lacks widespread adoption of such signing mechanisms.

To address this issue, Xia et al. [31] proposed a blockchain-based architecture for SBOM sharing that uses verifiable credentials to enable secure, tamper-resistant distribution. Their approach leverages the immutability and decentralized trust of blockchain systems to ensure that any tampering attempts would be detectable. By enabling selective disclosure and cryptographic assurance, their architecture also enhances trust without forcing unnecessary exposure of sensitive information.

Xia et al.'s work highlights that protecting SBOM integrity is crucial to mitigating risks to organizations that rely on them for security assurance. Without strong mechanisms to guard against tampering, SBOMs cannot fulfill their intended role in securing software supply chains.

## V. DISCUSSION

SBOMs are increasingly viewed as foundational tools for securing SSCs. However, the results of this review reveal a fragmented and evolving research landscape. While the literature confirms that SBOMs are being adopted and applied across several SSC security use cases, particularly vulnerability management and transparency, significant barriers remain. These barriers stem from technical limitations in tooling and standards, organizational and economic disincentives, and concerns over usability, accuracy, and risk. By analyzing these challenges through the lens of ISO/IEC 25019:2023's Quality in Use model, we are able to systematically assess not only

the functional gaps in SBOM implementations but also the experiential and risk-based shortcomings that affect stakeholder trust, acceptance, and adoption. This section synthesizes key findings, highlights important research gaps, offers practical takeaways for stakeholders, and outlines areas where future work is urgently needed.

### A. Gaps

Although the number of publications discussing SBOMs in the context of SSC security is increasing, several critical gaps persist. First, no papers in our review applied machine learning (ML) to SBOM data for SSC security purposes. This is notable given the potential of ML to analyze SBOMs for patterns that indicate security risks, predict exploitability, or automate triage of vulnerabilities. This represents a promising yet entirely unexplored research direction.

Second, we find no studies that use SBOMs in conjunction with software quality assurance (QA) techniques to model or measure SSC security. Given that SBOMs contain rich metadata about component provenance, dependencies, and usage, there is clear potential to extend QA frameworks with SBOM-informed metrics and evaluation pipelines.

Third, while several papers call attention to practical challenges in SBOM tooling and usage, very few studies critically evaluate the quality in use of SBOMs themselves. Our application of the ISO/IEC 25019:2023 QIU model reveals that the vast majority of SBOM limitations fall into three recurring categories: lack of trustworthiness (e.g., in analysis outputs, tamper resistance, or completeness), poor usability (e.g., in generation, interpretation, or sharing), and insufficient suitability for key security tasks (e.g., risk assessment, vulnerability triage). These categories are rarely analyzed holistically in the reviewed literature, suggesting the need for more rigorous empirical evaluations of SBOM effectiveness from the perspective of end users and adopters.

### B. Implications for Practitioners and End Users

For practitioners, our findings suggest that while SBOMs can improve SSC security, their current implementations often fall short of expectations. Issues such as high false positive rates, lack of exploitability context, and divergent analysis results from popular tools severely undermine user confidence. SBOMs are not yet turn-key solutions, effective use often requires substantial manual validation, tool tuning, and supplementary context such as VEX statements.

Organizations aiming to adopt SBOMs should approach implementation as a long-term process requiring investment in tooling, integration, and policy. Key priorities should include: choosing tools that support continuous SBOM updates; standardizing formats across internal teams; establishing processes for secure SBOM sharing; and developing policies for interpreting SBOM-derived vulnerability alerts.

For end users, particularly those consuming third-party software, our review underscores the importance of verifying SBOM quality. Many SBOMs are incomplete, outdated, or generated with low-fidelity tools. Relying blindly on provided

SBOMs without assessing their coverage, recency, or trustworthiness can create a false sense of security.

### C. Future Work

There are several clear directions for future research. First, the use ML with SBOM data for SSC security purposes is completely absent from the literature. SBOMs could serve as structured, interpretable inputs to ML models that predict SSC risk, detect anomalies, or assist in vulnerability prioritization.

Second, we see an opportunity to apply software QA and assurance techniques to systematically evaluate SBOM quality. For instance, test coverage metrics could be adapted to measure SBOM completeness, or fault injection methods could be used to assess how tools handle edge cases in SBOM generation.

Third, more work is needed to explore automation of exploitability context through tools like VEX, as well as hybrid SBOM + runtime telemetry approaches that could enable dynamic risk modeling. Similarly, future research should explore mechanisms to reduce false positives in vulnerability matching, perhaps through improved heuristics or ML classifiers.

Finally, we recommend continued development and benchmarking of SBOM tools using public datasets. Comparative studies that evaluate tools not only on generation fidelity but also on their QIU attributes, such as usability, trustworthiness, and maintainability, would provide crucial guidance to practitioners and advance SBOM research maturity.

## VI. THREATS TO VALIDITY

As with any systematic literature review, our study faces several potential threats to validity that must be considered when interpreting the findings. We discuss these threats in terms of construct validity, content validity, internal validity, and external validity; based on the classification scheme of Cook, Campbell and Day [54] and of Campbell et al. [55].

Construct and content validity refer to whether our study accurately captures what it aims to measure, in this case, the state of SBOM use and barriers in SSC security research. To mitigate this threat, we used a carefully designed search strategy based on the PICO method and screened all papers according to inclusion and exclusion criteria aligned with our research questions. However, the terminology surrounding SSC security and SBOMs is still emerging, and it is possible that some relevant papers were excluded due to variations in terminology or incomplete metadata. Additionally, we rely on the authors of each primary study to accurately describe their work and SBOM-related practices; we do not independently verify their claims.

Internal validity refers to cause and effect relationships between independent and dependent variables [56]. To support consistency and reduce bias, multiple co-authors were involved in the screening process. Each abstract was reviewed independently, and disagreements were resolved by full-text examination and consensus. While manual coding introduces some subjectivity, we believe it also allowed us to deeply

engage with the content and context of each paper, particularly when classifying SBOM barriers. Nonetheless, some papers may have been over- or under-coded due to ambiguous phrasing or limited methodological detail.

External validity refers to the generalizability of our findings. Although we drew from major digital libraries (IEEE Xplore, ACM Digital Library, Engineering Village) and systematically applied our search across them, our final corpus includes only 40 papers. This reflects the narrowness of current academic work on SBOMs for SSC security. We believe our dataset is representative of the literature to date, but recognize that gray literature and proprietary industry practices may not be captured. These could offer different insights into SBOM adoption and application in real-world settings.

By acknowledging these limitations, we aim to clarify the scope of our findings and support the reliability of this review. Future work may expand on this study by incorporating gray literature, automating parts of the screening process for greater scalability, or updating the review as the SBOM landscape evolves.

## VII. CONCLUSION

Software Bill of Materials (SBOMs) are increasingly viewed as critical for securing software supply chains (SSCs). This systematic literature review synthesizes current academic work on the adoption of SBOMs to bolster SSC security, identifies key applications and barriers, and highlights gaps in the existing research landscape.

Our findings confirm that SBOMs are being actively explored and applied in SSC security for tasks such as vulnerability management, risk assessment, and transparency. However, adoption remains constrained by a wide range of technical, organizational, and usability challenges. To better understand these challenges, we map the identified barriers to adoption to the ISO/IEC 25019:2023 Quality in Use (QIU) model [4]. This framework allows us to classify barriers according to their impact on usability, trustworthiness, suitability, and other quality dimensions relevant to real-world SBOM implementation.

Importantly, we identify several key gaps in the literature. No studies to date explore the application of machine learning to SBOM data for SSC security. Similarly, SBOMs have not yet been used in conjunction with software quality assurance techniques to assess or model SSC risks. Additionally, while many papers raise concerns about tooling quality and usability, few empirically evaluate SBOM tools from the perspective of QIU.

These gaps represent promising avenues for future research. As the SSC security landscape continues to evolve, SBOMs will likely play an increasingly central role. Our review offers a structured foundation for advancing both the academic understanding and practical application of SBOMs, and provides actionable insights for researchers, tool developers, and practitioners aiming to improve software supply chain resilience.

## VIII. ACKNOWLEDGMENTS

This research was conducted with support from the U.S. Department of Homeland Security (DHS) Science and Technology Directorate (S&T) under contract 70RSAT22CB0000005. Any opinions contained herein are those of the author and do not necessarily reflect those of DHS S&T.

## REFERENCES

- [1] T. Hastings and K. R. Walcott, "Continuous verification of open source components in a world of weak links," in *2022 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*. IEEE, 2022, pp. 201–207.
- [2] J. Martínez and J. M. Durán, "Software supply chain attacks, a threat to global cybersecurity: Solarwinds' case study," *International Journal of Safety and Security Engineering*, vol. 11, no. 5, pp. 537–545, 2021.
- [3] National Telecommunications and Information Administration, "The minimum elements for a software bill of materials (sbom)," Jul. 2021, accessed: 2024-12-3. [Online]. Available: <https://www.ntia.doc.gov/report/2021/minimum-elements-software-bill-materials-sbom>
- [4] *ISO/IEC 25019:2023(E): Systems and Software Engineering-Systems and Software Quality Requirements and Evaluation (SQuaRE)-Quality-in-use Model*, International Organization for Standardization Standard, 2023.
- [5] V. Sehgal and P. Ambili, "A taxonomy and survey of software bill of materials (sbom) generation approaches," 01 2024, pp. 40–51.
- [6] L. A. Jaatun, S. Marie Sørlien, R. Borgeonkar, S. Taylor, and M. G. Jaatun, "Software bill of materials in critical infrastructure," in *2023 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*, 2023, pp. 319–324.
- [7] N. Zahan, "Software supply chain risk assessment framework," in *2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 2023, pp. 251–255.
- [8] "Guidelines for performing systematic literature reviews in software engineering," Keele University and Durham University Joint Report, Tech. Rep. EBSE 2007-001, 07 2007. [Online]. Available: [https://www.elsevier.com/\\_data/promis\\_misc/525444systematicreviewsguide.pdf](https://www.elsevier.com/_data/promis_misc/525444systematicreviewsguide.pdf)
- [9] P. Ladisa, H. Plate, M. Martinez, and O. Barais, "Sok: Taxonomy of attacks on open-source software supply chains," in *2023 IEEE Symposium on Security and Privacy (SP)*, 2023, pp. 1509–1526.
- [10] T. Stalnakier, N. Wintersgill, O. Chaparro, M. Di Penta, D. M. German, and D. Poshvanyk, "Boms away! inside the minds of stakeholders: A comprehensive study of bills of materials for software systems," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: <https://doi.org/10.1145/3597503.3623347>
- [11] B. Xia, T. Bi, Z. Xing, Q. Lu, and L. Zhu, "An empirical study on software bill of materials: Where we stand and the road ahead," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 2630–2642.
- [12] Linux Foundation, "The state of software bill of materials (sbom) and cybersecurity readiness," <https://www.linuxfoundation.org/tools/the-state-of-software-bill-of-materials-sbom-and-cybersecurity-readiness>, accessed: 2024-11-09.
- [13] B. Kloeg, A. Y. Ding, S. Pellegrini, and Y. Zhauniarovich, "Charting the path to sbom adoption: A business stakeholder-centric approach," in *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*, ser. ASIA CCS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 1770–1783. [Online]. Available: <https://doi.org/10.1145/3634737.3637659>
- [14] S. Nocera, S. Romano, M. D. Penta, R. Francese, and G. Scanniello, "Software bill of materials adoption: A mining study from github," in *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2023, pp. 39–49.
- [15] Z. Sun, Z. Quan, S. Yu, L. Zhang, and D. Mao, "A knowledge-driven framework for software supply chain security analysis," ser. CCEAI '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 267–272. [Online]. Available: <https://doi.org/10.1145/3640824.3640866>
- [16] S. L. Eggers, B. R. Morgan, D. N. Christensen, T. B. Simon, and E. S. Bauer, "Proceedings of the 13th nuclear plant instrumentation, control & human-machine interface technologies (npic&hmit 2023)," in *Proceedings of the 13th Nuclear Plant Instrumentation, Control & Human-Machine Interface Technologies (NPIC&HMIT 2023)*, Knoxville, TN, July 15–20 2023, pp. 885–894.
- [17] N. K. Tran, S. Pallevatta, and M. A. Babar, "An empirically grounded reference architecture for software supply chain metadata management," in *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, ser. EASE '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 38–47. [Online]. Available: <https://doi.org/10.1145/3661167.3661212>
- [18] J. L. Sommer, M. M. Lund, N. Cibir, and M. Albano, "A method and platform for security advisory dissemination leveraging web3 technologies," in *2023 IEEE International Conference on Blockchain (Blockchain)*, 2023, pp. 265–272.
- [19] L. Bottner, A. Hermann, J. Eppler, T. Thüm, and F. Kargl, "Evaluation of free and open source tools for automated software composition analysis," ser. CSCS '23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: <https://doi.org/10.1145/3631204.3631862>
- [20] W. Wu, P. Wang, L. Zhao, and W. Jiang, "An intelligent security detection and response scheme based on sbom for securing iot terminal devices," in *2023 IEEE 11th International Conference on Information, Communication and Networks (ICIN)*, 2023, pp. 391–398.
- [21] E. O'Donoghue, A. M. Reinhold, and C. Izurieta, "Assessing security risks of software supply chains using software bill of materials," in *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering - Companion (SANER-C)*, 2024, pp. 134–140.
- [22] C. Soto-Valero, D. Tiwari, T. Toady, and B. Baudry, "Automatic specialization of third-party java dependencies," *IEEE Transactions on Software Engineering*, vol. 49, no. 11, pp. 5027–5045, 2023.
- [23] A. Chaora, N. Ensmenger, and L. J. Camp, "Discourse, challenges, and prospects around the adoption and dissemination of software bills of materials (sboms)," in *2023 IEEE International Symposium on Technology and Society (ISTAS)*, 2023, pp. 1–4.
- [24] T. M. S. do Amaral and J. J. C. Gondim, "Integrating zero trust in the cyber supply chain security," in *2021 Workshop on Communication Networks and Power Systems (WCNPS)*, 2021, pp. 1–6.
- [25] V. Ashwal, S. Finster, and A. Dawoud, "Llm-based vulnerability sourcing from unstructured data," in *2024 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, 2024, pp. 634–641.
- [26] T. Bi, B. Xia, Z. Xing, Q. Lu, and L. Zhu, "On the way to sboms: Investigating design issues and solutions in practice," vol. 33, no. 6, Jun. 2024. [Online]. Available: <https://doi.org/10.1145/3654442>
- [27] A. Mooduto, E. Rijanto, and G. C. Pamuji, "Optimization of software development automation via cicd, dependency track, and aws code-pipeline integration," in *2023 International Conference on Informatics Engineering, Science & Technology (INCITEST)*, 2023, pp. 1–7.
- [28] Q. Lu, L. Zhu, X. Xu, J. Whittle, D. Zowghi, and A. Jacquet, "Responsible ai pattern catalogue: A collection of best practices for ai governance and engineering," vol. 56, no. 7, Apr. 2024. [Online]. Available: <https://doi.org/10.1145/3626234>
- [29] R. A. Martin, "Visibility & control: Addressing supply chain challenges to trustworthy software-enabled things," in *2020 IEEE Systems Security Symposium (SSS)*, 2020, pp. 1–4.
- [30] S. Gupta and S. Vadlamudi, "Open-source software security challenges and policies for cloud enterprises," in *2023 3rd International Conference on Intelligent Communication and Computational Techniques (ICCT)*, 2023, pp. 1–6.
- [31] B. Xia, D. Zhang, Y. Liu, Q. Lu, Z. Xing, and L. Zhu, "Trust in software supply chains: Blockchain-enabled sbom and the aibom future," in *Proceedings of the 2024 ACM/IEEE 4th International Workshop on Engineering and Cybersecurity of Critical Systems (EnCyCrIS) and 2024 IEEE/ACM Second International Workshop on Software Vulnerability*, ser. EnCyCrIS/SVM '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 12–19. [Online]. Available: <https://doi.org/10.1145/3643662.3643957>
- [32] C. Okafor, T. R. Schorlemmer, S. Torres-Arias, and J. C. Davis, "Sok: Analysis of software supply chain security by establishing secure design properties," in *Proceedings of the 2022 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses*, ser. SCORED'22. New York, NY, USA: Association for Computing Machinery, 2022, p. 15–24. [Online]. Available: <https://doi.org/10.1145/3560835.3564556>

- [33] P. J. Caven, S. R. Gopavaram, and L. J. Camp, "Integrating human intelligence to bypass information asymmetry in procurement decision-making," in *MILCOM 2022 - 2022 IEEE Military Communications Conference (MILCOM)*, 2022, pp. 687–692.
- [34] B. Hassanshahi, T. N. Mai, A. Michael, B. Selwyn-Smith, S. Bates, and P. Krishnan, "Macaron: A logic-based framework for software supply chain security assurance," ser. SCORED '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 29–37. [Online]. Available: <https://doi.org/10.1145/3605770.3625213>
- [35] T. Lee, H. Park, and H. Lee, "Autometric: Towards measuring open-source software quality metrics automatically," in *2023 IEEE/ACM International Conference on Automation of Software Test (AST)*, 2023, pp. 47–55.
- [36] M. Mounesan, H. Siadati, and S. Jafarikhah, "Exploring the threat of software supply chain attacks on containerized applications," in *2023 16th International Conference on Security of Information and Networks (SIN)*, 2023, pp. 1–8.
- [37] T. Uehara, Y. Kanemoto, and H. Nomura, "Security transparency assurance technology for analysis and visualization of software components," *NTT Technical Review*, pp. 67 – 70, 2023//, cyberattacks;security transparency assurance technology;software components;software supply chain;supply chain security risks;updating software;.
- [38] D. E. Hyeon, J. H. Park, and H. Y. Youm, "A secure firmware and software update model based on blockchains for internet of things devices using sbom," in *2023 18th Asia Joint Conference on Information Security (AsiaJCIS)*, 2023, pp. 53–58.
- [39] A. Ferraiuolo, R. Behjati, T. Santoro, and B. Laurie, "Policy transparency: Authorization logic meets general transparency to prove software supply chain integrity," ser. SCORED'22. New York, NY, USA: Association for Computing Machinery, 2022, p. 3–13. [Online]. Available: <https://doi.org/10.1145/3560835.3564549>
- [40] Y. Du, O. Alrawi, K. Snow, M. Antonakakis, and F. Monrose, "Improving security tasks using compiler provenance information recovered at the binary-level," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 2695–2709. [Online]. Available: <https://doi.org/10.1145/3576915.3623098>
- [41] D. Brooks, "How to proactively use an sbom for vulnerability monitoring," Cybersecurity and Infrastructure Security Agency (CISA), 2023, available online at CISA's official website. [Online]. Available: <https://www.cisa.gov>
- [42] Cybersecurity and Infrastructure Security Agency (CISA), "Software Bill of Materials (SBOM)," <https://www.cisa.gov/sbom>, 2024, accessed: 2024-12-2.
- [43] Cisco Blogs, "Demonstrating Transparency Through Software Bill of Materials (SBOM)," <https://blogs.cisco.com/security/demonstrating-transparency-through-software-bill-of-materials-sbom>, 2024, accessed: 2024-12-2.
- [44] Joint Task Force Transformation Initiative, "SP 800-207: Zero Trust Architecture," National Institute of Standards and Technology (NIST), Gaithersburg, MD, USA, Tech. Rep., 2020. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-207.pdf>
- [45] D. Landoll, *The Security Risk Assessment Handbook: A Complete Guide for Performing Security Risk Assessments*, 3rd ed. CRC Press, 2021.
- [46] N. I. of Standards and Technology, "Software supply chain security in devsecops ci/cd pipelines," National Institute of Standards and Technology, Special Publication 800-204D, 2024.
- [47] S. Torres-Arias, D. Geer, and J. S. Meyers, "A viewpoint on knowing software: Bill of materials quality when you see it," *IEEE Security & Privacy*, vol. 21, no. 6, pp. 50–54, 2023.
- [48] M. Balliu, B. Baudry, S. Bobadilla, M. Ekstedt, M. Monperrus, J. Ron, A. Sharma, G. Skoglund, C. Soto-Valero, and M. Wittlinger, "Challenges of producing software bill of materials for java," *IEEE Security & Privacy*, vol. 21, no. 6, pp. 12–23, 2023.
- [49] D. Bifulco, S. Nocera, S. Romano, M. Di Penta, R. Francese, and G. Scanniello, "On the accuracy of github's dependency graph," ser. EASE '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 242–251. [Online]. Available: <https://doi.org/10.1145/3661167.3661175>
- [50] M. F. Rabbi, A. I. Champa, C. Nachuma, and M. F. Zibran, "Sbom generation tools under microscope: A focus on the npm ecosystem," in *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing*, ser. SAC '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 1233–1241. [Online]. Available: <https://doi.org/10.1145/3605098.3635927>
- [51] G. Dalia, C. A. Visaggio, A. Di Sorbo, and G. Canfora, "Sbom overture: What we need and what we have," in *Proceedings of the 19th International Conference on Availability, Reliability and Security*, ser. ARES '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: <https://doi.org/10.1145/3664476.3669975>
- [52] S. Yu, W. Song, X. Hu, and H. Yin, "On the correctness of metadata-based sbom generation: A differential analysis approach," in *2024 54th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2024, pp. 29–36.
- [53] A. Halbritter and D. Merli, "Accuracy evaluation of sbom tools for web applications and system-level software," in *Proceedings of the 19th International Conference on Availability, Reliability and Security*, ser. ARES '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: <https://doi.org/10.1145/3664476.3670926>
- [54] T. D. Cook, D. T. Campbell, and A. Day, *Quasiexperimentation: Design & Analysis Issues for Field Settings*. Boston, MA: Houghton Mifflin, 1979.
- [55] D. T. Campbell, J. C. Stanley, and N. L. Gage, *Experimental and Quasi-experimental Designs for Research*. Boston, MA: Houghton Mifflin, 1963.
- [56] C. Kaya, "Internal validity: A must in research designs," *Educational Research and Reviews*, vol. 10, no. 2, pp. 111–118, 2015.